Equational and Inductive Reasoning for Maude in Athena

Mateo Sanabria[1][0000−0003−3407−9792], Carlos Varela[2][0000−0003−4708−3109], Camilo Rocha[3][0000−0003−4356−7704], and Nicolás Cardozo[1][0000−0002−1094−9952]

[1] Universidad de los Andes, Bogotá , Colombia
[2] Rensselaer Polytechnic Institute, Troy, USA
[3] Pontificia Universidad Javeriana, Cali, Colombia

Abstract. In the rewriting logic framework, equational-based specifications are used to define deterministic functional behavior, abstract data types, and canonical representations of data. These specifications include a (possibly order-sorted) signature and equations interpreted modulo structural axioms, such as associativity, commutativity, and identity. While equational rewriting provides a powerful basis for execution and symbolic reasoning, it does not by itself offer native support for inductive or deductive reasoning. This paper presents maude2athena, a framework that systematically translates Maude's equational theories into Athena, a theorem proving language designed to support natural deduction proofs over many-sorted first-order logic specifications, including inductive reasoning, equational chaining, case-based reasoning, and proofs by contradiction. The translation supports induction-based reasoning modulo structural axioms with parametric induction rules; it faithfully encodes membership equational logic in a many-sorted setting without exponential blowup under reasonable conditions. This approach preserves the semantics of the original specification, while ensuring that the translation remains compact and amenable to deductive reasoning. This work helps bridge the gap between model checking and theorem proving, enabling formal verification efforts that can benefit from both of these approaches.

Keywords: Deductive reasoning · Inductive reasoning · Membership equational logic · Reasoning modulo axioms · Model checking and theorem proving · Maude · Athena

1 Introduction

Rewriting Logic [1] has established itself as a powerful semantic and specification framework for modeling computational systems, supporting both executable specifications and rigorous equational reasoning. Within this framework, *equational-based specifications* play a central role in defining deterministic functional behavior, abstract data types, and canonical representations of data. Languages such as Maude [2] provide strong support for this style of specification through membership equational logic, offering subsort relations, operator overloading, and rewriting modulo structural axioms such as associativity, commutativity, and identity. While rewriting logic is a logical framework that can be used for symbolic reasoning, including reasoning over its own specifications [3], Maude currently offers limited support for interactive theorem proving, especially for proofs that require explicit inductive reasoning or fine-grained proof control. In contrast, interactive theorem provers such as Athena [4] provide robust environments for natural-deduction reasoning in many-sorted first-order logic, together with strong support for induction and proof structuring. However, Athena lacks native support for the order-sorted features that are essential to many Maude specifications. As a consequence, fully-fledged Maude specifications cannot be directly reused within Athena without a careful reconciliation of their underlying logical foundations.

This paper presents maude2athena, a framework that systematically translates Maude functional modules into Athena modules, enabling equational and inductive reasoning over Maude specifications within Athena's interactive proof environment. The translation is grounded in a semantics-preserving mapping from membership equational logic to many-sorted first-order logic, in which subsort relations are made explicit through cast operators and coherence axioms [5]. To ensure scalability and avoid exponential blowup, the approach relies on the notion of strictly sensible order-sorted algebras [5, 6], which guarantees a linear and unambiguous translation in the presence of operator overloading.

A central challenge in this translation is the loss of native structural induction when order-sorted datatypes are flattened into many-sorted domains. Although this transformation preserves equational reasoning, it removes the inductive structure that is essential for proving many correctness properties. To address this issue, Athena is extended with *parametric structural induction primitives* that reconstruct the induction principles implicit in the original Maude specification. This design allows different induction schemes to be instantiated as needed, enabling inductive proofs over translated domains without resorting to encodings of operational semantics or meta-level reasoning. In this paper, this parametric framework is instantiated with structural induction for sufficiently complete equational specifications [7]; other induction principles can be accommodated as well, such as structural induction over the oriented equations of a terminating equational theory.

By bridging executable equational specifications in Maude with interactive inductive proofs in Athena, maude2athena enables a compositional workflow in which specification, execution, and proof coexist within a single formal development. The proposed translation preserves the equational theory of the original specification while remaining compact and amenable to both interactive and automated reasoning. The approach is validated through theoretical results establishing the correctness of the translation, as well as through a non-trivial case study involving the verification of a compiler specification that relies heavily on subsorting and inductive reasoning.[4]

2 Preliminaries

This section presents an overview on rewriting logic in Maude, and theorem proving in Athena.

2.1 Rewriting Logic and Maude

Rewriting logic [1] is built on top of membership equational logic [8] as the underlying equational formalism. Maude [2] is a high-performance logical framework and specification language based on rewriting logic. Membership equational logic supports sorts, equations, and membership axioms that characterize the elements of a sort. While the full language extends these characteristics with rewrite rules to represent transitions in concurrent systems, this paper focuses on the translation of the functional (equational) sub-language.

Membership equational logic is defined over a many-kinded signature (K, F), where K is a set of kinds and $F = \{F_{w,k}\}_{(w,k)\in K^*\times K}$ is a family of function symbols typed over those kinds. In addition to kinds, the logic considers a K-indexed family of sorts $S = \{S_k\}_{k\in K}$, which allows for ascribing sorts to terms using the notation $t : s$ (term t has sort s). Thus, a complete signature in membership equational logic is a triple $\Sigma = (K, F, S)$. The sorts S are equipped with a partial order $\leq$ (subsorting), and operators in F are defined over these sorts. This view associates the

[4] The maude2athena framework is available at `https://github.com/FLAGlab/Maude2Athena`.

kinds K as the connected components of the subsort relation $\leq$. Since an order-sorted algebra can be canonically mapped to a membership algebra [1, 9], the order-sorted setting is used throughout this paper. A membership equational theory is a pair (Σ, E), combining a signature Σ with a set of sentences E (conditional equations and membership axioms). Equality modulo E, denoted $=_E$, is the finest congruence on the term algebra $T_\Sigma(X)$ that satisfies the Horn clauses in E. That is, for any terms $t, u \in T_\Sigma(X)$, $t =_E u$ iff $E \vdash t = u$. Semantic structures of equational theories are called *algebras*. The expression T_Σ is the *sorted ground term algebra* of Σ and $T_\Sigma(X)$ the *sorted term algebra* of Σ with variables in X. Likewise, the quotient structure $T_{\Sigma/E}$ is the *initial algebra* of (Σ, E) meaning that two terms $t, u \in T_\Sigma$ are in the same equivalence class iff $t =_E u$. This is naturally extended to the algebra $T_{\Sigma/E}(X)$ of terms with variables in X modulo the equations E.

Maude partitions the specification into a set of structural axioms A (such as associativity, commutativity, and identity) and a set of functional equations E'. Consequently, an equational theory is understood as $(\Sigma, E' \cup A)$, where rewriting is performed modulo the structural axioms A by orienting the equations from left to right. In Maude, functional modules correspond to membership equational theories and define data types and their operations. Equations are treated as simplification rules applied left to right. By assuming *sort-decreasingness*, *ground confluence*, and *operational termination*, repeated application of the oriented equations reduces a term to its canonical form [10, 11]. Furthermore, it is assumed that an equational theory $(\Sigma, E' \cup A)$ is *sufficiently complete* with respect to a subsignature $\Omega \subseteq \Sigma$ meaning that, for any sort $s \in S$ and term $t \in T_{\Sigma,s}$, there is a term $u \in T_{\Omega,s}$ such that $t =_E u$.

Listing 1.1 presents a functional module specification of Peano numbers. The `PEANO` module is delimited by the Maude keywords `fmod` and `endfm`. Sorts are declared with the `sort` keyword, and subsort relations use `subsorts` together with the symbol `<` (Lines 2–3). Operators are declared with `op`; each declaration specifies the operator name, its sort signature, and an optional set of attributes that declare the structural set of axioms for the equational theory. Particularly, the attribute `ctor` marks constructors for the sort. The elements of the sort `Even` are defined recursively by a conditional equation (Line 8), and the Peano axioms are introduced as equations (Lines 9–10) defining the set E for the equational theory.

```
1  fmod PEANO is
2    sort NzNat Even Nat .
3    subsorts NzNat Even < Nat .
4    op zero : -> Even [ctor] .
5    op s_ : Nat -> NzNat [ctor] .
6    op _+_ : Nat Nat -> Nat .
7    vars N M : Nat .
8    cmb s s N : Even if N : Even .
9    eq N + zero = N .
10   eq N + s M = s (N + M) .
11 endfm
```

Listing 1.1. Definition of Peano numbers in Maude.

2.2 Theorem Proving in Athena

Athena [4] is a dual deduction and computation language: users can define *procedures*, which abstract over computations (i.e., lambda calculus abstractions in functional programming); and *methods*, which abstract over Fitch-style natural deductions [12, 13]. When evaluating procedures, if

successful, Athena can return values (of different types), or else, it can diverge or result in an error. When evaluating methods, if successful, Athena produces logically sound theorems—modulo its assumption base—as sentences in many-sorted first order logic [14]. Athena has been effectively used to reason about concurrent programming—particularly, the actor model [15, 16], and about safety-critical cyber-physical systems [17, 18].

Athena's proofs have a natural deduction style. Logical sentences can either be asserted into the assumption base or proven as theorems using Athena's deduction tools, with a soundness guarantee that any proven theorem is a logical consequence of sentences in the assumption base. Athena performs automatic sort-checking to prevent ill-sorted expressions in specifications and allows theorems to be introduced at an abstract level by encapsulating proofs in parameterized methods, which can be instantiated to prove different specializations of the abstract theorems. It defines two syntactic categories: an *expression*, which represents a computation, and a *deduction*, a logical argument such as a proof. A valid deduction always concludes with a *sentence*, *i.e.,* the conclusion of the proof.

Sorts form the foundation of Athena's deductive language, classifying all terms in its sentences. Sorts can be defined as *domains*, *structures*, and *datatypes*, each providing different mechanisms for constructing elements and each assuming different primitive axioms.

Domains are sorts that describe the sets of objects to be modeled. A *datatype* is a special kind of domain that is *inductively generated*, meaning that every element of the domain can be built up in a finite number of steps by applying *constructors* of the datatype. Beyond providing syntactic convenience over domains, datatypes can be assumed to follow *free-generation axioms* for their elements, ensuring that different constructor applications create different elements (*no-confusion* axioms), and that every element is represented by some constructor application (*no-junk* axioms).

Structures, just like regular datatypes, are inductively generated by their constructors. The only difference is that the constructors might not be injective, so that the same constructor applied to two distinct sequences of arguments might result in the same value.

The keyword `domain` introduces domains in Athena. Listing 1.2 presents a definition for Peano numbers based on three domains: `Nat, Even, NzNat`, with function symbols `zero,S,plus`, defined using the `declare` instruction. Note that, alternatively, Peano numbers can be defined using datatypes as: `datatype Nat := zero | (s Nat)`.

```
domains Nat, Even, NzNat
declare zero:[]         -> Even
declare s:   [Nat]      -> NzNat
declare plus:[Nat Nat] -> Nat [+]
define  [n m]            := [?n:Nat ?m:Nat]
assert  Plus-zero-axiom := (forall n .  (zero + n) = n)
assert* Plus-s-axiom    := ((n + (s m)) = (s (n + m)))
```

Listing 1.2. Peano example definition using domains.

The datatype definition provides free-generation axioms, ensuring that different constructor applications yield distinct elements. In contrast, the domain based approach offers more expressiveness by allowing sort refinements, like `Even` and `NzNat`, which enable more precise sort specifications. The version using domains is inspired by languages with subsort capabilities, such as Maude, and leverages Athena's many-sorted foundation to provide fine-grained sort control.

The semantics of Athena defines how phrases F (expressions or deductions) are evaluated within an *evaluation context*, a four-tuple $\langle \rho, \beta, \sigma, \gamma \rangle$ representing the logical and computational environment in which expressions are interpreted, where:

- ρ is the execution environment, a computable function that maps any given identifier I either to a value V or to a special *unbound* token.
- β is the assumption base, a finite set of sentences.
- σ is the store, a computable function that maps any natural number (representing a memory location) to a value (the location's contents) or to a special *unassigned* token.
- γ is a set of symbols with their associated signatures, and a collection of sort constructors (with arities). γ also includes information on whether a given sort constructor is a datatype or structure, and if so, which function symbols are its constructors.

The result of evaluating a phrase F in an evaluation context $\langle \rho, \beta, \sigma, \gamma \rangle$ is one of the following:

1. A pair (V, σ') consisting of a value V and a store σ', where V is the output of the evaluation and σ' reflects any side effects accumulated during the evaluation.
2. A pair consisting of an error message and a store σ', indicating the occurrence of an error during the computation.
3. Nontermination.

Sentences are added into the assumption base using `assert` or `assert*`, where `assert*` uses implicit universal quantification for all free variables, before inserting the sentence into the assumption base. Lines 6–7 in Listing 1.2 add the properties of Peano numbers into the assumption base.

Once sentences are introduced into the assumption base, they can be used in deductive reasoning. Further, the datatypes and structures definitions allow deductions of theorems that are not derivable from the free-generation axioms alone, by using the `by-induction` method. For example, assuming that `Nat` follows a datatype definition, Listing 1.3 displays the induction proof for the associativity property for the plus operator, Line 2.

In general, the method `by-induction` requires the proof of the property for all base cases (*zero* in the case of Peano, Lines 5–8) and the proof for all inductive definitions (S for `Nat` datatype, Lines 9–16). Additionally, this induction proof showcases one of the most useful Athena methods: `chain` (Lines 6–8 and Lines 12–16). `chain` allows equational chaining based proofs, where each step is labeled with its justification.

```
1  define [p q r] := [?p:Nat ?q:Nat ?r:Nat]
2  define plus_associative := (forall p q r . ((q + r) + p) = (q + (r + p)))
3  conclude plus_associative
4  by-induction plus_associative {
5     zero =>  pick-any q r
6             (!chain [((q + r) + zero)
7                     --> (q + r)          [Plus-zero-axiom]
8                     <-- (q + (r + zero)) [Plus-zero-axiom])
9     | (s p) => let {induction-hypothesis :=
10                  (forall ?q ?r . (?q + ?r) + p = ?q + (?r + p))}
11            pick-any q r
12            (!chain [ ((q + r) + (s p))
13                    --> (s ((q + r) + p))   [Plus-s-axiom]
14                    --> (s ((q + (r + p)))) [induction-hypothesis]
15                    <-- (q + (s (r + p)))   [Plus-s-axiom]
16                    <-- (q + (r + (s p)))  ])
17 }
```

Listing 1.3. Deduction of associativity property using datatype induction.

3 From Maude to Athena

At the heart of the proposed approach, lies the need to translate the order-sorted structure of terms of an equational theory in Maude to the many-sorted setting of equational theories in Athena. This section presents such a translation as a function from Maude modules to Athena theories. Translation of membership (of terms in sorts) is accomplished by using predicates in Athena.

3.1 Strictly Sensible Order-Sorted Signatures

The translation provides a linear translation of *strictly sensible* algebras to many-sorted algebras [5]. The key idea of the translation is to add an equivalence relation called core equality to the translated many-sorted structures. By defining this relation, the complexity of translating a strictly sensible order-sorted algebra to a many-sorted one is reduced and the translated many-sorted algebra equations only increase by a very small amount of new equations. Algebras definitions are lifted to signatures to keep the presentation of their results concise.

Fix a membership signature $\Sigma = (K, F, S)$, with kinds K, function symbols F, and sorts S, and a poset $(S, \leq)$ over the sorts. For a function symbol $g : s_1 \times \cdots \times s_n \to s$, $\text{target}(g) = s$ denotes its result sort. In what follows, assume $f, f', f'' \in F$ and, for a fixed $n \in \mathbb{N}$ and $s_i, s'_i, s''_i \in S$ for $0 \leq i < n$, where $f : s_0 \times \cdots \times s_n \to s$; $f' : s'_0 \times \cdots \times s'_n \to s'$; $f'' : s''_0 \times \cdots \times s''_n \to s''$.

Argument compatibility captures the idea of overloaded function symbols that can be treated uniformly as their pair-wise argument positions share a common supersort. Strongly sensible signatures have argument compatible function symbols in which the target sorts are the same.

Definition 1. *Two overloaded function symbols f and f' are called argument compatible, denoted $ac(f, f')$, iff $(\forall\, i \mid 0 \leq i \leq n \ : s_i \equiv_{\leq} s'_i)$, where $s_i \equiv_{\leq} s'_i$ is a predicate expressing that s_i and s'_i are in the same connected component of $(\Sigma, \leq)$. The signature Σ is strongly sensible iff $(\forall\, f, f' \mid ac(f, f') \ : s = s')$.*

On the other hand, maximal argument-bounding signatures ensure that every function symbol has a representative among its compatible function symbols, thereby defining one representative for each class of argument compatible function symbols.

Definition 2. *The signature Σ is maximal argument-bounding iff:*

$$(\forall\, f \mid (\exists\, f'' \mid (\forall\, f' \mid ac(f, f') \ : (\forall\, i \mid 0 \leq i \leq n \ : s'_i \leq s''_i)))).$$

A *strictly sensible* signature is a signature that is both strongly sensible and maximal argument-bounding. Strictly sensible signatures play a central role in the translation from order-sorted equational theories in Maude to many-sorted theories in Athena. It rules out ambiguous cases of function symbol overloading and ensures that the translation remains well-defined and systematic. In contrast to more general translations (e.g., [19]), which can produce non-linear or ambiguous mappings in the presence of unrestricted overloading, requiring strict sensibility guarantees that every function symbol has a single, consistent representative in the translated signature. In particular, strict sensibility ensures that each function symbol has a unique representative in the translated signature, allowing the order-sorted algebra to be flattened into a many-sorted representation without loss of soundness or consistency.

In the rest of the paper, it is assumed that any signature Σ is strictly sensible.

3.2 The Translation Function

Let $\mathcal{E} = (\Sigma, E \cup A)$ be a membership equational theory, with signature (K, F, S) and poset of sorts $(S, \leq)$, and $\Omega \subseteq \Sigma$ be a constructor subsignature for $\mathcal{E}$. This section defines the mapping $\mathcal{E} \mapsto (\beta, \gamma)$ as the function *tr*, where β is an Athena assumption base and γ is its symbol set. The ρ execution environment and σ store are omitted from the mapping because they are orthogonal to the generated Athena representation. The behavior of *tr* is driven primarily by the subsort relation $\leq$ and by the constructors Ω.

Definition 3 (Translation Function tr**).** *Let $\mathcal{E} = (\Sigma, E \cup A)$ be a membership equational theory with signature $\Sigma = (K, F, S)$, sort poset $(S, \leq)$, and constructor subsignature $\Omega \subseteq \Sigma$. The function $\mathrm{tr} : \mathcal{E} \to (\beta, \gamma)$ is defined by the following five components:*

(i) Sort translation (**tr_S**). *For each sort $s \in S$:*

$$\mathrm{tr}_S(s) = \begin{cases} \texttt{datatype}(s,\ \{c \in \Omega \mid \mathrm{target}(c) = s\}) & \text{if } s \notin \mathrm{dom}(\leq) \cup \mathrm{ran}(\leq) \text{ and } \exists\, c \in \Omega. \\ \texttt{domain}(s) & \text{otherwise} \end{cases}$$

(ii) Function symbol translation (**tr_F**). *Let $[f]_{\mathrm{ac}}$ denote the argument compatibility class of $f \in F$. For each class $[f]_{\mathrm{ac}}$, select the maximal representative f^* (guaranteed to exist by strict sensibility), where $f^* \colon s_1 \times \cdots \times s_n \to s$, and set*

$$\gamma \ \cup\ \left\{\, f^* \colon [\mathrm{tr}_S(s_1) \cdots \mathrm{tr}_S(s_n)] \to \mathrm{tr}_S(s) \,\right\} \ \mapsto\ \gamma.$$

For each $s \leq s'$ in $(S, \leq)$, set

$$\gamma \ \cup\ \left\{\, \mathrm{Cast}_{s \to s'} \colon [\mathrm{tr}_S(s)] \to \mathrm{tr}_S(s') \,\right\} \ \mapsto\ \gamma.$$

Additionally, for each $f^* : s \times s \to s$ in γ with structural attributes in A, let x, y, z be fresh variables of sort $\mathrm{tr}_S(s)$ and set

$$\beta \ \cup\ \left\{ \begin{array}{ll} f^*(f^*(x,y),z) = f^*(x, f^*(y,z)) & \text{if } \texttt{assoc} \in \mathrm{attr}(f), \\ f^*(x,y) = f^*(y,x) & \text{if } \texttt{comm} \in \mathrm{attr}(f), \\ f^*(\mathrm{tr}_T(e), x) = x,\ f^*(x, \mathrm{tr}_T(e)) = x & \text{if } \texttt{id}{:}e \in \mathrm{attr}(f) \end{array} \right\} \ \mapsto\ \beta.$$

(iii) Term translation (**tr_T**). *For each $t \in T_\Sigma(X)$:*

$$\mathrm{tr}_T(t) = \begin{cases} x & \text{if } t = x \in X \\ f^*(\mathrm{tr}_T(t_1), \ldots, \mathrm{tr}_T(t_n)) & \text{if } t = f(t_1, \ldots, t_n) \end{cases}$$

where f^* is the maximal representative of $[f]_{\mathrm{ac}}$ and, at each argument position i where the subterm has sort s'_i with $s'_i \leq s_i$ and $s'_i \neq s_i$, $\mathrm{tr}_T(t_i)$ is wrapped in the appropriate cast chain $\mathrm{Cast}_{s_j \to s_{j+1}}(\cdots(\mathrm{tr}_T(t_i))\cdots)$ so that the result has sort $\mathrm{tr}_S(s_i)$.

(iv) Equation translation (**tr_E**). *For each equation $l = r$ in $E \cup A$, set*

$$\beta \ \cup\ \left\{\, \mathrm{tr}_T(l) = \mathrm{tr}_T(r) \,\right\} \ \mapsto\ \beta.$$

For each composable pair $s \leq s' \leq s''$, add the core equality*:*

$$\beta \ \cup\ \left\{\, \mathrm{Cast}_{s' \to s''}(\mathrm{Cast}_{s \to s'}(x)) = \mathrm{Cast}_{s \to s''}(x) \,\right\} \ \mapsto\ \beta,$$

ensuring all casting paths between the same source and target sorts are identified.

(v) Membership translation **(**tr_M**).** *For each sort s appearing in a membership axiom, set*

$$\gamma \cup \left\{ \mathrm{is_s}\colon[\mathrm{tr}_S(\mathrm{kind}(s))] \to \texttt{Boolean} \right\} \mapsto \gamma.$$

Then, for each unconditional membership $t : s$ and each conditional membership $t : s$ if P, set $\beta \cup \{\mathrm{tr}_M(\cdot)\} \mapsto \beta$, where:

$$\mathrm{tr}_M(t : s) = \mathrm{is_s}(\mathrm{tr}_T(t)) \qquad \mathrm{tr}_M(t : s \textit{ if } P) = (\mathrm{tr}_T(P) \Rightarrow \mathrm{is_s}(\mathrm{tr}_T(t)))$$

The five components of *tr* interact as a pipeline: tr_S determines how each sort is represented in Athena; tr_F uses these representations to declare function and cast symbols and to assert the structural axioms of A; tr_T relies on both to translate terms, inserting casts where implicit subsorting must become explicit; tr_E and tr_M then lift the term translation to equations and membership axioms, respectively. To make each step concrete, the rest of this section walks through *tr* component by component on two variants of the Peano numbers specification: the simple module in Listing 1.4 (no subsort relation) and the richer module in Listing 1.1 (with subsorts `NzNat` and `Even`), showing the corresponding Athena output in Listing 1.5 and Listing 1.6.

```
fmod PEANO is
    sort Nat .
    op zero : -> Nat [ctor] .
    op s_ : Nat -> Nat [ctor] .
    op _+_ : Nat Nat -> Nat .
    vars N M : Nat .
    eq N + zero = N .
    eq N + s M = s (N + M) .
endfm
```

Listing 1.4. Simple Peano module in Maude.

```
1 module PEANO {
2     datatype Nat := zero  | (s Nat)
3     declare + : [Nat Nat] -> Nat
4     define [N M] := [?N:Nat ?M:Nat]
5     assert* (N + zero = N)
6     assert* (N + s M = s (N + M))
7 }
```

Listing 1.5. Simple Peano module in Athena.

Sorts. Through tr_S, Maude sorts are translated into Athena domains or datatypes. Whenever a sort $s \in S$ satisfies that it does not appear in the subsort relation $\leq$ and that there are constructors with target sort s, *tr* translates it to a datatype. In any other case, sorts are translated to domains. For instance, `zero` and `s_` in Listing 1.4 are function symbols (with target sort `Nat`) defined with the `ctor` attribute. Therefore, tr_S uses their sort signatures to define the corresponding Athena datatype, as shown in Line 2 of Listing 1.5. The remaining defined function symbols with target sort s are translated with the `declare` keyword. This is the case for the `_+_` function symbol, which is translated into Line 3 of Listing 1.5.

The treatment is different when the sort s is part of the subsort relation, as is the case in Listing 1.1. This module introduces `NzNat` and `Even` as subsorts of `Nat`. In this case, the previous translation case no longer applies. For sorts related through the subsort relation (`NzNat`, `Even`, and `Nat`) the translation function tr_S instantiates each sort as a plain Athena domain (Line 2 in Listing 1.6). The process then proceeds through the four remaining translation components (tr_F, tr_T, tr_E, tr_M): (1) function symbols and introducing explicit cast function symbols, (2) terms using casting function symbols, (3) equations into Athena assertions using the transformed terms, and (4) conditional membership axioms using term translation and predicate introduction.

Function Symbols. In the simple module (Listing 1.4), no subsort relation exists, so tr_F declares each function symbol directly: `_+_` becomes the Athena declaration in Line 3 of Listing 1.5. In the richer module (Listing 1.1), the subsorts introduce overloading. The three operators `zero`, `s_`, and `_+_` are grouped by tr_F into argument compatibility classes; for each class the unique maximal representative is selected and declared in γ (Lines 3–5 of Listing 1.6). Additionally, the two subsort pairs `Even` $\leq$ `Nat` and `NzNat` $\leq$ `Nat` produce two cast symbols (Lines 6–7 of Listing 1.6).

Terms and Equations. In the simple module, no subsort relation exists, so tr_T leaves terms unchanged and tr_E translates each equation directly into an Athena assertion. In the richer module, implicit coercions must be made explicit. Consider the first equation in Listing 1.1 (Line 9): because the argument has sort `Even` while the operator expects `Nat`, tr_T inserts a cast (Line 9 of Listing 1.6). The second equation (Line 10 of Listing 1.1) requires casts on both the argument and the result (Line 11 of Listing 1.6). Because the subsort poset of this module has no composable triple $s \leq s' \leq s''$, no core equalities are generated; in specifications with longer subsort chains, tr_E would add the corresponding transitivity assertions to β.

Memberships. Membership judgements have no native counterpart in Athena's many-sorted logic. The component tr_M bridges this gap by introducing a unary predicate for each sort that appears in a membership axiom; the predicate's domain is the kind of that sort, so it can be applied to any term of the right kind. In the richer Peano module, the conditional membership in Line 8 of Listing 1.1 is translated into an implication guarded by a membership predicate (Lines 12–13 of Listing 1.6). Here, tr_M chooses the kind of the target sort as the predicate's domain (*i.e.,* the topmost supersort in the connected component). The simple module contains no membership axioms, so tr_M produces no output for it. The complete translation of Listing 1.1 is shown in Listing 1.6.

```
1  module PEANO{
2    domains NzNat Even Nat
3    declare s : [Nat] -> NzNat
4    declare zero : [] -> Even
5    declare + : [Nat Nat] -> Nat
6    declare Cast_Even_to_Nat : [Even] -> Nat
7    declare Cast_NzNat_to_Nat : [NzNat] -> Nat
8    define [N M] := [?N:Nat ?M:Nat]
9    assert* ((+ N (Cast_Even_to_Nat zero)) = N)
10   assert* ((+ N (Cast_NzNat_to_Nat (s M)))
11         = (Cast_NzNat_to_Nat (s (+ N M))))
12   declare is_even : [Nat] -> Boolean
13   assert* ((is_even N) ==> (is_even s s N))  }
```

Listing 1.6. Peano Athena Module

Finally, the correctness of the translation is stated with respect to equational provability.

Theorem 1 (Equational Provability). *Let $\mathcal{E} = (\Sigma, E \cup A)$ be a membership equational theory. If Σ is strictly sensible and $\mathcal{E}$ is sufficiently complete w.r.t. $\Omega \subseteq \Sigma$, then for any $t, u \in T_{\Sigma,k}(X)$ of the same kind k, the following two sentences are equivalent:*

1. $t =_{\mathcal{E}} u$
2. $\mathrm{tr}_T(t) =_\beta \mathrm{tr}_T(u)$, i.e., $\mathrm{tr}_T(t)$ *and* $\mathrm{tr}_T(u)$ *are provably equal in Athena under the theory* $\mathrm{tr}(\mathcal{E}) = (\beta, \gamma)$ *with the inference rules of many-sorted equational logic.*

Proof. The equational fragment of the translation (components tr_S, tr_F, tr_T, and tr_E) instantiates the strictly sensible order-sorted–to–many-sorted mapping [5]. The membership component tr_M introduces predicate symbols and assertions of the form $\mathrm{is_s}(\mathrm{tr}_T(t))$ and $\mathrm{tr}_T(P) \Rightarrow \mathrm{is_s}(\mathrm{tr}_T(t))$; because is_s is a predicate symbol, no assertion produced by tr_M can have the form $t' = u'$ for terms t', u' of a non-Boolean sort, so tr_M does not introduce new equalities between translated terms. Hence, the equational content of β is precisely the image of E under tr_E, the structural axioms of A under tr_F, together with the core equalities.

◇ $t =_{\mathcal{E}} u \Rightarrow \mathrm{tr}_T(t) =_\beta \mathrm{tr}_T(u)$
The proof proceeds by structural induction on the derivation of $t =_{\mathcal{E}} u$, case-splitting on the last inference rule applied.

- **Axiom instantiation.** Suppose $t =_{\mathcal{E}} u$ is obtained by applying a substitution θ to an equation $l = r$ in $E \cup A$, so that $t = \theta(l)$ and $u = \theta(r)$. By tr_E, the translated equation $\mathrm{tr}_T(l) = \mathrm{tr}_T(r)$ is in β, universally quantified over all free variables. Because tr_T is defined compositionally and the cast insertion at each argument position depends only on the sort of the subterm and the expected sort of the position, tr_T commutes with substitution: for each variable x, define $\hat{\theta}(x) = \mathrm{tr}_T(\theta(x))$; then $\mathrm{tr}_T(\theta(t')) = \hat{\theta}(\mathrm{tr}_T(t'))$ for any term t', possibly up to core equalities in β that identify different casting paths to the same target sort. Hence, $\hat{\theta}(\mathrm{tr}_T(l)) =_\beta \hat{\theta}(\mathrm{tr}_T(r))$ follows by instantiation of the universally quantified axiom in β.
- **Reflexivity, symmetry, transitivity.** These are inference rules of many-sorted equational logic and thus hold natively in Athena's deductive system.
- **Replacement.** Suppose $t = C[t']$ and $u = C[u']$ for a context C and $t' =_{\mathcal{E}} u'$, and assume by the induction hypothesis that $\mathrm{tr}_T(t') =_\beta \mathrm{tr}_T(u')$. The translation tr_T maps C to a context C' in the many-sorted signature γ: each function symbol in C is replaced by its maximal representative f^*, and explicit casts are inserted at every argument position where a subsort coercion is needed. Because each cast symbol $\mathrm{Cast}_{s\to s'}$ is a function symbol in γ, the standard congruence rule of many-sorted equational logic guarantees that if $a =_\beta b$ then $\mathrm{Cast}_{s\to s'}(a) =_\beta \mathrm{Cast}_{s\to s'}(b)$. Moreover, if t' and u' require different casting chains to reach the expected sort of the hole in C', the core equalities (added by tr_E for every composable pair $s \le s' \le s''$) ensure that both chains yield equal results in β. Hence, $\mathrm{tr}_T(C[t']) = C'[\mathrm{tr}_T(t')] =_\beta C'[\mathrm{tr}_T(u')] = \mathrm{tr}_T(C[u'])$ by congruence in many-sorted equational logic.

Since every inference rule used in the derivation of $t =_{\mathcal{E}} u$ is preserved, $\mathrm{tr}_T(t) =_\beta \mathrm{tr}_T(u)$.

◇ $\mathrm{tr}_T(t) =_\beta \mathrm{tr}_T(u) \Rightarrow t =_{\mathcal{E}} u$
It suffices to show that β is a conservative extension of the image of $E \cup A$ with respect to equalities between translated terms. The axioms of β fall into three classes:
(a) translated equations $\mathrm{tr}_T(l) = \mathrm{tr}_T(r)$ for each $l = r$ in $E \cup A$;
(b) core equalities $\mathrm{Cast}_{s'\to s''}(\mathrm{Cast}_{s\to s'}(x)) = \mathrm{Cast}_{s\to s''}(x)$ for every composable pair $s \le s' \le s''$;
(c) membership assertions produced by tr_M.
As argued above, class (c) consists entirely of Boolean-sorted sentences and cannot produce new equalities between non-Boolean terms.
For classes (a) and (b), the argument proceeds as follows:

- **Unambiguous operator mapping.** Strict sensibility guarantees that each argument compatibility class $[f]_{\mathrm{ac}}$ has a unique maximal representative f^*. The translation maps every f in the same class to the *same* f^* and *omits* all others. Because the selection is unique, no two originally distinct function symbols are merged by the translation, therefore no invalid equalities between terms arise from the operator mapping.

- **Conservative cast equalities.** Each cast symbol $\mathrm{Cast}_{s \to s'}$ is a fresh uninterpreted function symbol in γ, distinct for each pair $s < s'$. The only axioms governing cast symbols are the core equalities in class (b), which encode the transitivity and path independence of the subsort relation. Because the subsort relation $\leq$ is already part of $\mathcal{E}$, the core equalities do not identify terms that are not already related by subsorting.
- **Isomorphism of initial algebras.** According to Li *et al.* [5], the strictly sensible mapping combined with the core casting equalities induces a bijection between the equivalence classes of the initial order-sorted algebra $T_{\Sigma/(E \cup A)}$ and those of the initial many-sorted algebra $T_{\gamma/\beta_{\mathrm{eq}}}$, where β_{eq} denotes the equational part of β (classes (a) and (b)). The sufficient completeness assumption ensures that every ground term reduces to a constructor term, which is needed for the bijection to be an isomorphism of algebras.

Therefore, if $\mathrm{tr}_T(t) =_\beta \mathrm{tr}_T(u)$, then $\mathrm{tr}_T(t)$ and $\mathrm{tr}_T(u)$ lie in the same equivalence class of $T_{\gamma/\beta_{\mathrm{eq}}}$. By the isomorphism, t and u belong to the same class in $T_{\Sigma/(E \cup A)}$, *i.e.,* $t =_{\mathcal{E}} u$.

4 Inductive Reasoning

The translation function *tr* from $(\Sigma, E \cup A)$ to (β, γ) presented in Section 3 enables equational reasoning over Maude specifications within Athena by preserving the underlying equational theory in a many-sorted first-order logic setting. However, equational reasoning alone is often insufficient to establish properties of interest, which typically require some form of inductive reasoning. This section explains how equational reasoning with $tr(\mathcal{E})$ in Athena can be extended with inductive reasoning based on parametric induction principles. The key idea is that for the induction principle to be applied, it must be instantiated using the structure of sorts in Σ associated with each connected component in the poset $(S, \leq)$. To illustrate this approach, this section presents a concrete instantiation of such a parametric induction principle, showing how structural induction can be recovered and applied in practice.

In Athena, there is native support for structural, constructor-based induction over datatypes, which allows properties to be proved by reasoning directly on the inductive structure generated by datatype constructors. However, the translation function *tr* systematically represents many Maude sorts as domains, rather than datatypes, in order to preserve the structure induced by subsorting in the many-sorted setting of Athena. As a consequence, Athena's built-in induction mechanisms are no longer directly applicable, and native datatype induction is insufficient for reasoning about the translated specifications. To address this limitation, the approach adopted here is to equip Athena with induction principles defined over domains, rather than relying solely on datatype induction. These principles are formulated in a parametric way, allowing different induction schemes to be instantiated depending on the structure of the sorts under consideration. In particular, this includes natural extensions of Athena's native structural induction to domains that arise from translated equational specifications.

More generally, an induction principle in this setting is realized by declaring a primitive method in Athena that explicitly encodes both the basis cases and the inductive cases associated with a given sort structure $(S, \leq)$. For a selected induction principle η, the primitive method is defined separately for each connected component of the sort hierarchy, reflecting the constructors and subsort relations that characterize that component. The basis cases correspond to the minimal elements or constructors of the component, while the inductive cases capture how the property is preserved by the relevant constructor applications. Once declared, such a primitive method can

be applied uniformly to predicates over the corresponding domain, thereby extending equational reasoning with inductive reasoning in a principled and modular way.

The following definition extends the translation function (Definition 3) with a sixth component, tr_η, that formalizes the generation of structural induction principles. It operates on the execution environment ρ—previously noted as orthogonal to the generated representation (β, γ), but now required to host the primitive methods that realize induction.

Definition 4 (Induction Method Translation tr_η). *Let $\mathcal{E}$, Σ, Ω, and $(S, \leq)$ be as in Definition 3. The translation function* tr *is extended with:*

(vi) Induction method translation (tr_η). *For each kind k in $(S, \leq)$, let $C = \{\, s \in S \mid \exists\, c \in \Omega.\ \mathrm{target}(c) = s \,\} \cap [k]_\leq$ be the set of sorts in the connected component of k that are directly generated by Ω-constructors. Define the* effective constructor set

$$\Omega_C^+ \;=\; \Omega_C \;\cup\; \{\, \mathrm{Cast}_{s \to s'} \mid s \notin C,\ s' \in C,\ s \leq s' \,\},$$

where $\Omega_C = \{c \in \Omega \mid \mathrm{target}(c) \in C\}$ and the second component adds subsort injection casts from sorts outside C into C. By construction, Ω_C^+ is jointly exhaustive: every constructor normal form of kind k is headed by some element of Ω_C^+. For each $c : s_1 \times \cdots \times s_n \to s$ in Ω_C^+, let $I_c = \{\, i \mid 1 \leq i \leq n,\ s_i \in C \,\}$ be the set of recursive argument positions. *The component tr_η generates a primitive method that takes a predicate $P : \mathrm{tr}_S(k) \to \texttt{Boolean}$ and encodes the obligations of $\eta_C(P)$ as follows.*

(a) Base-case sentences.

(1) For each constant $c :\to s$ in Ω_C^+ (0-arity, so $I_c = \varnothing$), define

$$b_c \;:=\; P\big(\mathrm{Cast}_{s \to k}(c)\big).$$

(2) For each constructor $c : s_1 \times \cdots \times s_n \to s$ in Ω_C^+ with $n \geq 1$ and $I_c = \varnothing$, define

$$b_c \;:=\; \forall\, x_1 \cdots x_n \,.\, P\big(\mathrm{Cast}_{s \to k}(c(x_1, \ldots, x_n))\big),$$

where each $x_i : \mathrm{tr}_S(s_i)$.

(b) Inductive-case sentences. *For each constructor $c : s_1 \times \cdots \times s_n \to s$ in Ω_C^+ with $I_c \neq \varnothing$, define the sentence*

$$h_c \;:=\; \forall\, x_1 \cdots x_n \,.\, \Big(\bigwedge_{i \in I_c} P\big(\mathrm{Cast}_{s_i \to k}(x_i)\big)\Big) \;\Rightarrow\; P\big(\mathrm{Cast}_{s \to k}(c(x_1, \ldots, x_n))\big),$$

where each $x_i : \mathrm{tr}_S(s_i)$. When $|I_c| = 1$, the conjunction reduces to a single hypothesis.

(c) Primitive method assembly. *Let $B = \{b_c\}$ and $H = \{h_c\}$ be the collected base-case and inductive-case sentences, and let $(o_1, \ldots, o_m)$ be an enumeration of all sentences in $B \cup H$. The primitive method for component C is added to the execution environment ρ as*

$$\rho \;\cup\; \big\{\, \texttt{prim-method}_C(P) \;:\; \texttt{check}_m \;\Rightarrow\; \forall\, x : \mathrm{tr}_S(k).\ P(x) \,\big\} \;\mapsto\; \rho,$$

where each obligation is verified sequentially through nested `check` *expressions:* $\texttt{check}_1$ *tests* $\texttt{holds?}(o_1)$ *and, upon success, enters* $\texttt{check}_2$*, which tests* $\texttt{holds?}(o_2)$*, and so on. In general, for $i = 1, \ldots, m$,* $\texttt{check}_i$ *tests* $\texttt{holds?}(o_i)$ *and proceeds to* $\texttt{check}_{i+1}$*; when $i = m$, the innermost check concludes $\forall\, x : \mathrm{tr}_S(k).\ P(x)$. If any* $\texttt{holds?}(o_i)$ *fails, the method signals an error.*

The full translation thus maps $\mathcal{E} \mapsto (\beta, \gamma, \rho)$, extending the original pair with the execution environment populated by the generated primitive methods.

As an example, consider the primitive method in Listing 1.7. It defines `nat-induction`, a structural induction principle for the `Nat` domain in Listing 1.6. It explicitly captures the two components of a standard induction proof. The first definition, labeled `basis`, corresponds to the sentence of the basis case over the constant constructor `Cast_Even_to_Nat zero`. The second definition, labeled `ic`, encodes the inductive case: it states that if the property holds for an arbitrary `Nat` element `x` (inductive hypothesis), then it needs to hold for its successor, `Cast_NzNat_to_Nat (s x)`. The `check` expression enforces both conditions. It first verifies that the basis case holds, and if so, proceeds to check the inductive case. This mechanism effectively restores structural induction for domains translated from Maude datatypes, allowing the proof of properties such as associativity or commutativity complementing the built-in `by-induction` method.

```
primitive-method (nat-induction property) :=
  let {
    basis := (property (Cast_Even_to_Nat zero));
    ic := (forall x (if (property x) (property (Cast_NzNat_to_Nat (s x)))))
    }
    check { (holds? basis) =>
              check {(holds? ic) => (forall x (property x))
                    | else => (error "Inductive step does not hold.")}
              | else => (error "Basis step does not hold.")}
```

Listing 1.7. Primitive induction method for Nat.

The following theorem establishes the soundness of inductive proofs carried out using the primitive methods generated by tr_η.

Theorem 2 (Soundness of Inductive Proofs). *Let $\mathcal{E} = (\Sigma, E \cup A)$ be a membership equational theory that is sufficiently complete a connected component of $(S, \leq)$ with kind k, and $P : \mathrm{tr}_S(k) \to$ `Boolean` a predicate. If the primitive method `prim-method`$_C(P)$ generated by tr_η (Definition 4) checks all base cases and inductive steps and concludes $\forall\, x : \mathrm{tr}_S(k).\ P(x)$, then for every ground term t of sort $s \in C$ in $T_{\Sigma,k}$, the property $P(\mathrm{tr}_T(t))$ holds in Athena under the theory $\mathrm{tr}(\mathcal{E}) = (\beta, \gamma, \rho)$.*

Proof. Assume, for contradiction, that there exists a ground term t of sort $s \in C$ in $T_{\Sigma,k}$ such that $P(\mathrm{tr}_T(t))$ does not hold, even though all obligations of `prim-method`$_C(P)$ have been checked. Because $\mathcal{E}$ is sufficiently complete, t reduces to a constructor normal form headed by some $c \in \Omega$ with $\mathrm{target}(c)$ in the connected component of k. If $\mathrm{target}(c) \in C$, then $c \in \Omega_C \subseteq \Omega_C^+$; otherwise c is an injection from an external sort $s \notin C$ with $s \leq s'$ for some $s' \in C$, so $c = \mathrm{Cast}_{s \to s'}$ belongs to the second component of Ω_C^+. In either case $c \in \Omega_C^+$. By Theorem 1, $P(\mathrm{tr}_T(t))$ holds if and only if P holds on the translated normal form, so it may be assumed without loss of generality that $t = c(t_1, \ldots, t_n)$ is already in constructor normal form. It proceeds by induction on the number of constructor applications in t.

Base case: ($I_c = \varnothing$). The constructor c has no argument positions of sorts in C, so t contains exactly one constructor application. The primitive method verified the base-case sentence b_c (Definition 4). Whether $c :\to s$ is a constant (in which case b_c directly asserts $P(\mathrm{Cast}_{s \to k}(c))$) or $c : s_1 \times \cdots \times s_n \to s$ with $n \geq 1$ and $I_c = \varnothing$ (in which case b_c is universally quantified and instantiates at $\mathrm{tr}_T(t_1), \ldots, \mathrm{tr}_T(t_n)$) the verified sentence b_c yields $P(\mathrm{tr}_T(t))$.

Inductive step: ($I_c \neq \varnothing$). For each $i \in I_c$, the subterm t_i is a ground term of sort $s_i \in C$ with strictly fewer constructor applications than t. By the induction hypothesis, $P(\mathrm{tr}_T(t_i))$ holds for every $i \in I_c$. The primitive method verified the inductive-case sentence h_c (Definition 4), which asserts that the conjunction of $P(\mathrm{Cast}_{s_i \to k}(x_i))$ for $i \in I_c$ implies $P(\mathrm{Cast}_{s \to k}(c(x_1, \ldots, x_n)))$. Instantiating at $\mathrm{tr}_T(t_1), \ldots, \mathrm{tr}_T(t_n)$ and applying modus ponens with the induction hypotheses yields $P(\mathrm{tr}_T(t))$.

In both cases $P(\mathrm{tr}_T(t))$ holds, contradicting the assumption. Since t was arbitrary, $P(\mathrm{tr}_T(t))$ holds for every ground term of kind k.

5 Case Study

The case study verifies inductive properties of a compiler for numeric expressions on a Stack Machine [4]. The case study relies heavily on order-sorted features of data types and includes structural axioms.[5] Listing 1.8 shows the Maude specification, defining three sorts: a source language of arithmetic expressions (`Exp`), a target instruction set (`Instr`/`Program`), and a stack machine (`Stack`).

```
fmod TOY-COMPILER is
  protecting INT .
  sort Exp .
  subsort Int < Exp .
  op _plus_ : Exp Exp -> Exp [ctor] .
  op _minus_ : Exp Exp -> Exp [ctor] .
  op _mult_ : Exp Exp -> Exp [ctor] .
  sorts Instr Program Stack .
  subsort Instr < Program .
  op push : Int -> Instr [ctor] .
  ops add sub mult : -> Instr [ctor] .
  op nil : -> Program [ctor] .
  op _++_ : Program Program -> Program [ctor assoc id: nil] .
  op empty : -> Stack [ctor] .
  op _::_ : Int Stack -> Stack [ctor] .
  op I : Exp -> Int .         --- interpreter
  op compile : Exp -> Program .
  op exec : Program Stack -> Stack .
  vars N N1 N2 : Int .  vars E1 E2 : Exp .
  var P : Program .     var S : Stack .
  eq I(N) = N .
  eq I(E1 plus E2) = I(E1) + I(E2) .
  --- I equations for minus, mult analogous
  eq compile(N) = push(N) .
  eq compile(E1 plus E2) =
     compile(E2) ++ compile(E1) ++ add .
  --- compile equations for minus, mult analogous
  eq exec(nil, S) = S .
  eq exec(push(N) ++ P, S) = exec(P, N :: S) .
  eq exec(add ++ P, N1 :: N2 :: S) =
```

[5] The complete Maude specification of the compiler with the translated equationally equivalent Athena program (with proofs) is available at: `https://github.com/FLAGlab/Maude2Athena`

```
31      exec(P, (N1 + N2) :: S) .
32    --- exec equations for sub, mult analogous
33    ...
34  endfm
```

Listing 1.8. Toy compiler in Maude (shorten).

The compiler specification relies on two order-sorted features. First, `Int < Exp` lets integers appear directly as expressions without an explicit injection constructor. Second, `Instr < Program` lets single instructions be used where programs are expected. The operator `_++_` carries the structural axioms `assoc` and `id: nil`.

To preserve the semantics of `Int < Exp`, the translation introduces explicit cast functions for every subsort relation in the sort poset, as explained in Section 3. maude2athena automatically inserts these casts. For example, the equations in lines Line 24 and Line 29 are translated to:

```
declare push : [Int] -> Instr
declare compile : [Exp] -> Program
declare Cast_Int_to_Exp : [Int] -> Exp
declare Cast_Instr_to_Program : [Instr] -> Program
assert* eq_4 := ((compile (Cast_Int_to_Exp N))
                    = (Cast_Instr_to_Program (push N)))
assert* eq_9 := ((exec (++ (Cast_Instr_to_Program (push N)) P) S)
                    = (exec P (:: N S)))
```

Since Athena requires unique function symbols and does not support Maude's mixfix notations natively, the tool flattens operators and resolves overloading. Specifically, the mixfix Maude operator `_plus_` is translated to a standard prefix function in Athena. If overloading collisions are detected, unique identifiers are generated based on sort signatures:

```
declare plus : [Exp Exp] -> Exp
declare ++ : [Program Program] -> Program
assert* ((compile (plus E1 E2)) =
         (++ (compile E2)
             (++ (compile E1) (Cast_Instr_to_Program add))))
define [_v1 _v2 _v3 _v4] :=
       [?_v1:Program ?_v2:Program ?_v3:Program ?_v4:Program]
assert* assoc_++ := ((++ (++ _v1 _v2) _v3) = (++ _v1 (++ _v2 _v3)))
assert* left_id_++ := ((++ nil _v4) = _v4)
assert* right_id_++ := ((++ _v4 nil) = _v4)
```

This listing also illustrates how the structural axioms A are handled. The operator `_++_` is declared with the attributes `assoc, id: nil`, meaning that rewriting is performed modulo associativity and identity. Because Athena has no built-in support for such axioms, tr_F translates each structural axiom into an explicit equational assertion: `assoc_++` encodes associativity, while `left_id_++` and `right_id_++` encode the left and right identity axioms for `nil`. In general, any combination of associativity, commutativity, and identity (ACU) attributes attached to operators in A is translated by tr_F into the corresponding set of universally quantified assertions in β.

The tool analyzes the signature to determine which sorts can be modeled as `datatypes` (allowing native induction) and which must be `domains`. In this example, the Maude `Stack` is translated as an Athena datatype:

```
datatype Stack := empty | (:: Int Stack)
```

However, Exp and Program are translated as domains to accommodate the subsorting relations that are otherwise incompatible with standard datatype constructors.

The result of this translation is a fully typed Athena module where the correctness of the compiler (`forall e . (exec (compile e)) = (I e)`) can be rigorously proven. The core verification goal is the following theorem, which asserts that compiling an expression `e` followed by a program `p` is equivalent to executing `p` on a stack with the evaluated result of `e`:[6]

```
(forall ?p ?s .   (exec (++ (compile ?e) ?p) ?s) =
                  (exec ?p (:: (I ?e) ?s)))
```

Note that this sentence cannot be asserted as a logical sentence because Exp is a domain, not a datatype. Instead, it is defined as a predicate over expressions. This enables the theorem to be passed as an argument to the induction method of choice. In this case, the predicate `correctness` is defined as:

```
declare correctness: [Exp] -> Boolean
assert* correctness_def := (iff (correctness ?e)
    (forall ?p ?s .
        (exec (++ (compile ?e) ?p) ?s) = (exec ?p (:: (I ?e) ?s))))
```

Proving that `(forall ?e . correctness ?e)` holds, brings to attention the challenges of reasoning over non-inductive domains and handling explicit subsorting.

Note that in Maude, Exp is defined via subsorting ($Int < Exp$), which prevents it from being translated into a standard Athena `datatype`. Consequently, Athena's native `by-induction` method cannot be applied. To overcome this, `maude2athena` automatically generates a custom **primitive method** that reconstructs the inductive principle of the original Maude sort. The generated method, `exp-induction`, takes the `correctness` predicate as an argument and strictly enforces that the user proves the inductive step for every constructor (e.g., `plus`, `mult`) before discharging the goal:

```
primitive-method (exp-induction property) :=
    let {
        basis_n  := (forall ?n (property (Cast_Int_to_Exp ?n)))
        ic_plus  := (forall ?e1 (forall ?e2
                        (if (and (property ?e1) (property ?e2))
                            (property (plus ?e1 ?e2)))));
        ic_minus := ...
        ic_mult  := ...
    }
    check { (holds? basis_n) =>   ...  }
```

The translation replaces implicit subsorting with explicit cast operators. This requires the proof script to handle these casts in the basis cases. For instance, the basis case for integers is not merely `forall n`, but rather `forall n` applied to the cast `Cast_Int_to_Exp`. The proof proceeds by unwrapping these casts using the axioms generated by the translation (*e.g.,* `eq_4`):

[6] The correctness property follows trivially as a corollary of this theorem by instantiating it to the empty program on an empty stack.

```
define basis_step := (forall n . correctness (Cast_Int_to_Exp n))
conclude basis_step
  pick-any n:Int p:Program s:Stack
    (!chain [ (exec (++ (compile (Cast_Int_to_Exp n)) p) s)
            = (exec (++ (Cast_Instr_to_Program (push n)) p) s)   [eq_4]
            = (exec p (:: n s))                                  [eq_9]
            ... ])
```

This explicit handling ensures that the proof is rigorous and mathematically consistent with the original order-sorted semantics. Finally, the proof is concluded by invoking the custom induction method, which aggregates the base case and inductive steps to discharge the main theorem: `(!exp-induction correctness)`.

6 Related Work

The formal verification of rewriting logic specifications typically involves a trade off between the high performance execution provided by environments like Maude and the interactive deductive capabilities of theorem provers like Athena [4], Isabelle [20], or Lean [21]. There are three main methodologies designed to address this challenge: (1) the development of native domain-specific theorem provers, (2) the application of automated symbolic reasoning techniques, and (3) the integration with general purpose proof assistants via translation.

The native approach, pioneered by the original Maude Inductive Theorem Prover (ITP) [2] and modernized by NuITP [22], offers powerful automation using variant-based reasoning directly at Maude's metalevel. While highly effective for discharging goals automatically without translation, tools like NuITP inherently generate proofs as metalevel search traces. In contrast, maude2athena translates specifications into Athena to leverage its natural-deduction framework, which closely mirrors standard mathematical reasoning [13]. Rather than competing on pure automation, the proposed approach emphasizes proof transparency and structure. Athena's built-in methods (equality chaining, structural induction, case analysis, and many others) allow proofs to be developed and read as structured mathematical arguments rather than execution traces or low-level tactic scripts, following human-like proof strategies [23]. Consequently, maude2athena occupies a complementary niche to native provers like NuITP: it targets verification scenarios where transparent, independently checkable, and human-readable proofs are the primary requirement.

A highly effective alternative to interactive deduction relies on automated symbolic methods. Maude integrates with SMT solvers to discharge proof obligations generated from specifications. Furthermore, techniques such as narrowing and its combination with SMT solving allow for the analysis of infinite state systems [24]. Reachability Logic and its associated tools also provide mechanisms to deductively reason about Maude programs [25, 26]. These automated methodologies are essential for verifying safety properties and analyzing state spaces, though they may not cover properties requiring complex inductive arguments or higher order reasoning.

The third strategy is to translate specifications to an external formalism to leverage its proof infrastructure. The Heterogeneous Tool Set (Hets) [27] treats Maude as an institution, enabling translations to Isabelle/HOL via CASL [28]. While theoretically robust, this path relies on complex encodings to simulate subsorting. Similarly, approaches to translate order-sorted algebras into many-sorted algebras have been explored to leverage standard theorem provers [5, 6]. The Maude2Lean framework [29] presents a direct translation from rewriting logic to the calculus of inductive constructions. Unlike the institutional approach of Hets, Maude2Lean translates the syntax of Maude

terms and the semantics of rewriting logic into inductive datatypes in Lean. This preserves the inductive structure of the logic, allowing native structural induction on terms and relations. However, such translations shift the first-order theorem proving problem to a different domain (e.g., type theory or higher order logic), sometimes incurring in a verbosity trade-off, as proofs become arguments about the translated semantics rather than direct reasoning about the domain model.

7 Concluding Remarks

This work presents maude2athena, a tool for translating order-sorted Maude specifications into the many-sorted first-order logic of Athena. The main contribution of this work is the reconciliation of symbolic reasoning and equational rewriting with the power of inductive reasoning, enabling specifications that leverage the best of both worlds. To the best of the authors' knowledge, this is the first implementation grounded in the theoretical translation proposed by Li *et al.* [5]. The proposed approach bridges the semantic gap between the two formalisms by making subsorting explicit via cast operators, modeling membership as predicates, and recovering structural induction through custom primitive methods. The maude2athena framework was validated using several examples. It includes the case study of a compiler specification, demonstrating the approach's ability to handle deep inductive structures and operator overloading, while preserving the equational consistency of the original system.

The future work focuses on two key areas. First, to enhance modularity by mapping Maude's built-in modules directly to Athena's native domains. Such integration would allow developers to leverage Athena's existing decision procedures and automated reasoning capabilities for standard data types. Second, extending maude2athena to handle rewrite rules to enable the verification of concurrent and distributed systems. The current version is restricted to functional modules, while rewrite rules are the primary mechanism for defining concurrent state transitions. Athena, being grounded in standard first-order logic, lacks native primitives for modeling the non-determinism and asynchrony inherent in such rules. Rather than constructing a new concurrency framework within Athena from scratch, the idea is to directly work on top of Talcott's formalization of Actor theories in rewriting logic [30]. By translating these pre-validated semantic structures directly into Athena, Maude's system modules can be mapped to logical transition relations. On the one hand, this would significantly reduce the formalization burden. On the other hand, it will effectively equip Athena with the capability to reason about temporal properties and distributed protocols without needing to manually reconstruct the underlying theory of concurrency.

Acknowledgements. The authors would like to thank the anonymous reviewers for their very insightful comments on an earlier version of this paper. They would like to also thank Konstantine Arkoudas for his insight on encoding structural induction over plain domains as a primitive method in Athena to preserve the inductive reasoning capabilities. Rocha acknowledges support from the SGR project PROMUEVA (BPIN 2021000100160) under the supervision of Minciencias (Ministerio de Ciencia Tecnología e Innovación, Colombia).

References

1. Meseguer, J.: Conditional rewriting logic as a unified model of concurrency. Theoretical computer science (1992)

2. Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., and Talcott, C.: All about Maude-a high-performance logical framework: how to specify, program, and verify systems in rewriting logic. Springer (2007)
3. Clavel, M., Meseguer, J., and Palomino, M.: Reflection in membership equational logic, many-sorted equational logic, Horn logic with equality, and rewriting logic. Theoretical Computer Science (2007)
4. Arkoudas, K. and Musser, D.: Fundamental proof methods in computer science: a computer-based approach. MIT Press (2017)
5. Li, L. and Gunter, E.L.: A Method to Translate Order-Sorted Algebras to Many-Sorted Algebras. In: Fourth International Workshop on Rewriting Techniques for Program Transformations and Evaluation. EPTCS, pp. 20–34 (2017)
6. Goguen, J.A. and Meseguer, J.: Order-sorted algebra I: Equational deduction for multiple inheritance, overloading, exceptions and partial operations. Theoretical Computer Science (1992)
7. Rocha, C. and Meseguer, J.: Constructors, Sufficient Completeness, and Deadlock Freedom of Rewrite Theories. In: Logic for Programming, Artificial Intelligence, and Reasoning, pp. 594–609. Springer (2010)
8. Bouhoula, A., Jouannaud, J.-P., and Meseguer, J.: Specification and proof in membership equational logic. Theoretical Computer Science (2000)
9. Meseguer, J.: Membership algebra as a logical framework for equational specification. In: Recent Trends in Algebraic Development Techniques, pp. 18–61. Springer (1998)
10. Durán, F. and Meseguer, J.: On the Church-Rosser and coherence properties of conditional order-sorted rewrite theories. The Journal of Logic and Algebraic Programming (2012)
11. Lucas, S. and Meseguer, J.: Operational termination of membership equational programs: the order-sorted way. Electronic Notes in Theoretical Computer Science (2009)
12. Arkoudas, K.: Denotational proof languages. PhD thesis, Massachusetts Institute of Technology (2000).
13. Arkoudas, K.: Simplifying Proofs in Fitch-Style Natural Deduction Systems. Journal of Automated Reasoning (2005)
14. Manzano, M.: Extensions of first-order logic. Cambridge University Press (1996)
15. Musser, D.R. and Varela, C.A.: Structured Reasoning About Actor Systems. In: Proceedings of the 2013 Workshop on Programming Based on Actors, Agents, and Decentralized Control. AGERE! 2013, pp. 37–48. ACM, Indianapolis, Indiana, USA (2013)
16. Varela, C.A.: "The AMST Language: Formal Verification and Execution of Actor Systems". In: Concurrent Programming, Open Systems and Formal Methods: Essays Dedicated to Gul Agha to Celebrate His Scientific Career. Cham: Springer Nature Switzerland, 2026, pp. 36–59.
17. Paul, S., McCarthy, C., Patterson, S., and Varela, C.: Formal verification of timely knowledge propagation in airborne networks. Science of Computer Programming (2025)
18. Paul, S., Cruz, E., Dutta, A., Bhaumik, A., Blasch, E., Agha, G., Patterson, S., Kopsaftopoulos, F., and Varela, C.: Formal Verification of Safety-Critical Aerospace Systems. IEEE Aerospace and Electronic Systems Magazine (2023)
19. Meseguer, J. and Skeirik, S.: Equational formulas and pattern operations in initial order-sorted algebras. Form. Asp. Comput. (2017)
20. Wenzel, M., Paulson, L.C., and Nipkow, T.: The Isabelle framework. In: International Conference on Theorem Proving in Higher Order Logics, pp. 33–38 (2008)
21. Moura, L.d. and Ullrich, S.: The Lean 4 theorem prover and programming language. In: International Conference on Automated Deduction, pp. 625–635 (2021)

22. Durán, F., Escobar, S., Meseguer, J., and Sapiña, J.: NuITP: An inductive theorem prover for equational program verification. In: International Symposium on Principles and Practice of Declarative Programming, pp. 1–11 (2024)
23. Bringsjord, S. and Govindarajulu, N.S.: Fundamental Proof Methods in Computer Science: A Computer-Based Approach, by Arkoudas and Musser, The MIT Press, Cambridge, USA, ISBN 978-0-262-03553-8. Theory and Practice of Logic Programming (2021)
24. Rocha, C., Meseguer, J., and Muñoz, C.: Rewriting modulo SMT and open system analysis. In: International Workshop on Rewriting Logic and its Applications, pp. 247–262 (2014)
25. Romero, M. and Rocha, C.: Symbolic Execution and Reachability Analysis Using Rewriting Modulo SMT for Spatial Concurrent Constraint Systems with Extrusion. In: NASA Formal Methods, pp. 435–451. Springer (2018)
26. Arias, J., Bae, K., Olarte, C., Ölveczky, P.C., Petrucci, L., and Rømming, F.: Symbolic analysis and parameter synthesis for time Petri nets using Maude and SMT solving. In: International Conference on Applications and Theory of Petri Nets and Concurrency, pp. 369–392 (2023)
27. Codescu, M., Mossakowski, T., Riesco, A., and Maeder, C.: Integrating Maude into HETS. In: International Conference on Algebraic Methodology and Software Technology, pp. 60–75 (2010)
28. Astesiano, E., Bidoit, M., Kirchner, H., Krieg-Brückner, B., Mosses, P.D., Sannella, D., and Tarlecki, A.: CASL: the common algebraic specification language. Theoretical Computer Science (2002)
29. Rubio, R. and Riesco, A.: Maude2Lean: Theorem proving for Maude specifications using Lean. Journal of Logical and Algebraic Methods in Programming (2022)
30. Talcott, C.: Actor theories in rewriting logic. Theoretical Computer Science (2002)